

清华大学电子工程系本科生核心课

数据与算法 - 第2讲

二元关系、抽象数据类型、线性表

冯伟 (fengwei@tsinghua.edu.cn)

2024年9月24日



提纲



- 二元关系
- 抽象数据类型
- 线性表

提纲



- 二元关系
- 抽象数据类型
- 线性表

为什么要学习二元关系

数理基础

数据与算法

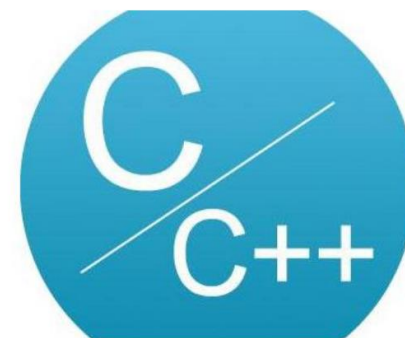
程序设计



修骨骼：“看你骨骼惊奇，是万中无一的练武奇才，我这儿有…”



练内功：把数据结构、算法等基础课程学好



学招式：学习编程语言（弹钢琴、炒菜），积累代码量



为什么要学习二元关系

在原子弹研究初期，苏联专家曾提供过一些技术数据。但是后来我国研究人员利用“九次计算”（一种解方程的模拟方法）得出的一个关键数据，却与苏联专家提供的技术指标不符合，引起争议。

1961年，理论物理和粒子物理学家周光召加入“九所”（原二机部第九研究所）。他从头至尾重算了一遍，开始怀疑：会不会专家给的数据错了？这是一个大胆的怀疑。一个没有搞过原子弹的人，否定专家给的数据，谈何容易？

周光召发挥了他**深厚的数理基础功底**，他从能量利用率入手，利用“最大功原理”证明了“九次计算”结果的正确性和苏方数据的不可能。

后来，使用研究人员计算的数据，我们掌握了原子弹研制的关键技术，问题也就迎刃而解。

——摘自中国工程院原副院长杜祥琬在第十三届创新中国论坛上的讲话



为什么要学习二元关系

在突破氢弹的过程中，研究人员用晶体管计算机对理论模型进行计算，每个时刻的计算结果都会打在一条纸带上，纸带上详细记录着该时刻的速度、加速度、压力、坐标等物理量。

研究人员就在计算机前盯着纸带上密密麻麻的数字，观察物理量的变化。有一天，于敏和我们正在看这些数据，突然于敏说：“这个量错了！”**一般来说，计算机算出来的数据，是很难靠人脑算出来的。**但于敏就发现了计算机算出的数据有问题——他看到某个物理量变化趋势的异常，他的物理学功底告诉他，这个数据不对。

大家开始排查，物理、数学、计算方面的专家齐动手，发现物理参数、方程没问题，计算编程、程序指令、程序逻辑也没错。但于敏坚持这其中一定有问题。当时的晶体管计算机是由装满晶体管的大柜子组成的，它们一个个摆在大厅里。

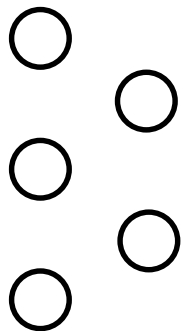
会不会是计算机出了问题？后来去请计算机专家检查。最后，还真就发现其中一个加法器的晶体管坏了。他们把坏掉的晶体管换下来，对那个物理量重新计算了一遍，才得出了正确的结果。计算机专家们非常惊讶：“你们从物理量出发，能找出计算机硬件的一个错误，实在令人佩服。”而于敏之所以能够质疑计算机，就缘于其在**理论物理学上打下的深厚基础**。

——摘自中国工程院原副院长杜祥琬在第十三届创新中国论坛上的讲话



用二元关系描述数据的逻辑结构

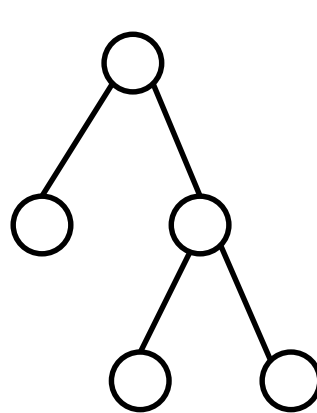
- 数据的基本单元称为数据元素，数据元素并不是孤立存在的，而是存在密切的联系（→信息、知识），因此可表示和描述客观事物
- 数据的逻辑结构：集合上的二元关系



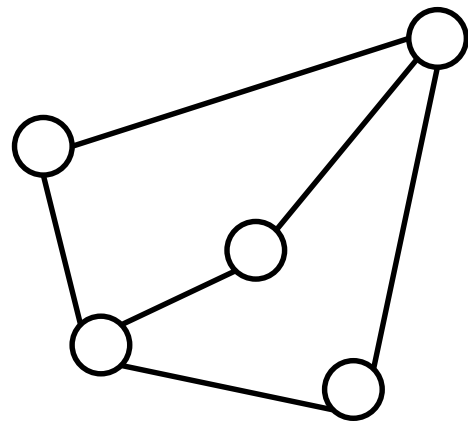
集合



线性结构



树结构



图结构



二元关系

- 数据元素之间的联系，归纳起来有三种，都可以借助“二元关系”进行描述
 - 一对一的联系：如线性结构
 - 一对多的联系：如树结构
 - 多对多的联系：如图结构
- **二元关系是描述数据之间关系的数学方法**
- 利用集合论中的符号体系，将各种数据集中数据元素之间的关系用数学的语言精确描述出来
- **为应对更为复杂、动态的数据结构问题提供基础**



二元关系

➤ 集合的笛卡尔积

- 集合 M, N : $M = \{x\}, N = \{y\}$
- 集合 M 和 N 的笛卡尔积, 记作: $M \times N$, 定义为:

$$M \times N = \{(x, y) \mid x \in M \text{ 且 } y \in N\}$$

➤ $M \times N$ 中的每个元素都是一个二元组, 称为有序对或者序偶

➤ 不满足交换律

➤ 例: $M = (a_1, a_2), N = (0, 1, 2)$ 则:

$$M \times N = \{(a_1, 0), (a_1, 1), (a_1, 2), (a_2, 0), (a_2, 1), (a_2, 2)\}$$



二元关系

- $M \times N$ 的任意一个子集 $R \subset M \times N$, 称为 **M 到 **N 的一个二元关系****
- 二元关系表示了集合 M 和 N 中元素之间的**某种相关性**
- 例：某学生学习语文, 数学, 外语, 表示为 $M = \{\text{语文}, \text{数学}, \text{外语}\}$ 功课的成绩分四个等级, 记作 $N = \{A, B, C, D\}$, 则该生成成绩的全部可能为:
 $M \times N = \{\langle \text{语文}, A \rangle, \langle \text{语文}, B \rangle, \langle \text{语文}, C \rangle, \langle \text{语文}, D \rangle, \langle \text{数学}, A \rangle, \langle \text{数学}, B \rangle, \langle \text{数学}, C \rangle, \langle \text{数学}, D \rangle, \langle \text{外语}, A \rangle, \langle \text{外语}, B \rangle, \langle \text{外语}, C \rangle, \langle \text{外语}, D \rangle\}$;
若 $R = \{\langle \text{语文}, B \rangle, \langle \text{数学}, A \rangle, \langle \text{外语}, D \rangle\}$,
 R 是 $M \times N$ 的一个子集, 它表示该生功课和成绩的对应关系。



二元关系的性质

在实际应用的大多数情形下，两个集合是相同的，称 R 是集合
 M ($M = N$) 上的一个二元关系

➤ **自反性**：对于每个 $a \in M$ ，有 $(a, a) \in R$

反自反性：对于所有 $a \in M$ ，有 $(a, a) \notin R$

➤ **对称性**：当 $(a, b) \in R$ 时，必有 $(b, a) \in R$

反对称性：当 $(a, b) \in R$ 且 $(b, a) \in R$ 时，则 $a = b$ ；

或者，如果 $(a, b) \in R$ 则 $(b, a) \notin R$

➤ **传递性**：当 $(a, b) \in R$ 且 $(b, c) \in R$ 时，必有 $(a, c) \in R$



二元关系举例

- 实数集合上的 $=$, $\leq$, $<$ 关系
 - $=$ 是自反的, 对称的, 传递的
 - $\leq$ 是自反的, 反对称的, 传递的
 - $<$ 是反自反的, 反对称的, 传递的
- 集合的 $=$, $\subseteq$, $\subset$ 的关系
- 父子关系, 同班同学关系
- 平面上三角形的全等关系, 相似关系

利用性质, 更方便、更准确地分析描述数据之间的关联规律和特点



等价关系

➤ 满足自反性，对称性和传递性，则称这个关系为等价关系

- 自反性：对于每个 $a \in M$ ，有 $(a, a) \in R$
- 对称性：当 $(a, b) \in R$ 时，必有 $(b, a) \in R$
- 传递性：当 $(a, b) \in R$ 且 $(b, c) \in R$ 时，必有 $(a, c) \in R$

➤ 例子

- 实数集合上的相等关系
- 集合的相等关系
- 同班同学关系
- 平面三角形集合中，三角形全等和相似关系



偏序关系

- 满足自反性，反对称性和传递性，则称这个关系为偏序关系
 - 自反性：对于每个 $a \in M$ ，有 $(a, a) \in R$
 - 反对称性：对于所有 $a, b \in M$ ，如果 $(a, b) \in R$ 和 $(b, a) \in R$ ，则 $a = b$
 - 传递性：当 $(a, b) \in R$ 且 $(b, c) \in R$ 时，必有 $(a, c) \in R$
- 例子
 - 实数集合上的小于等于关系 “ $\leq$ ”
 - 子集包含关系 “ $\subseteq$ ”



拟序关系（严格偏序关系）

➤ 满足反自反性，反对称性和传递性，则称为拟序关系

- 反自反性：对于每个 $a \in M$ ，有 $(a, a) \notin R$
- 反对称性：对于所有 $a, b \in M$ ，如果 $(a, b) \in R$ 和 $(b, a) \in R$ ，则 $a = b$
- 传递性：当 $(a, b) \in R$ 且 $(b, c) \in R$ 时，必有 $(a, c) \in R$

➤ 例子

- 实数集合上的小于关系 “ $<$ ”
- 子集真包含关系 “ $\subset$ ”



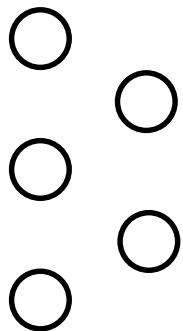
全序关系

- 设 R 是集合 M 上的一个偏序关系，如果集合 M 中的任意两个元素 a 和 b 都是可比的，也就是说或者有 aRb 成立，或者有 bRa 成立，则称 R 是集合 M 上的全序关系
- 实数集合 R 上定义的“ $\leq$ ”是全序关系
- 定义了全序关系 R 的集合 M ，称为全序集



二元关系→数据的逻辑结构

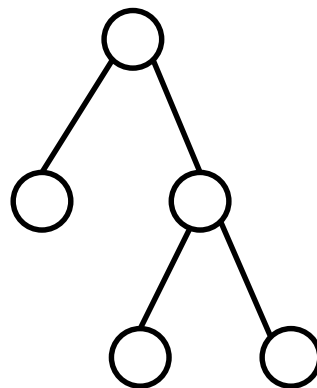
- 从二元关系出发可以定义数据的逻辑结构，在宏观尺度上反应数据元素之间在逻辑上的联系；定义二元组 $B = (D, R)$ ，其中 D 为数据元素的有限集合， R 为 D 上二元关系的有限集合
- 数据逻辑结构的要素：①数据元素的集合，②集合上二元关系



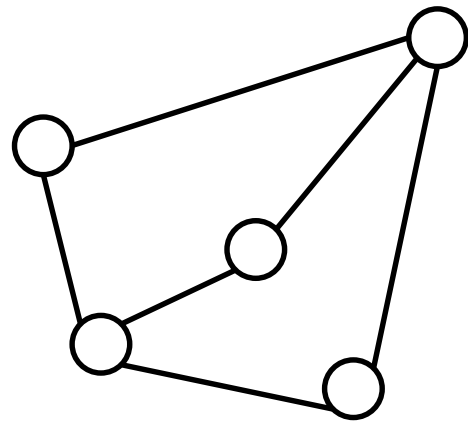
集合



线性结构



树结构



图结构

提纲



- 二元关系
- 抽象数据类型
- 线性表



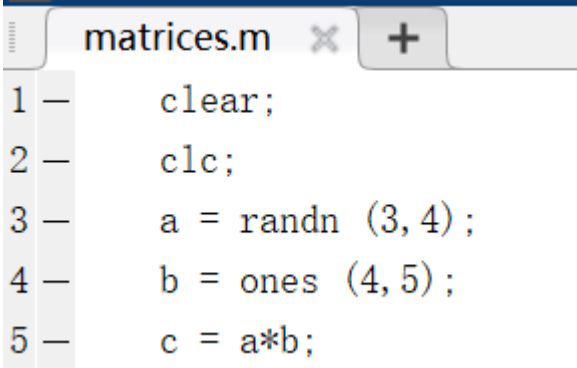
为什么需要数据类型

■ 程序片段：矩阵相乘

```
for( i = 0; i < n; i++) {  
    for(j = 0; j < n; j++) {  
        c[i][j] = 0;  
        for( k = 0; k < n; k++)  
            c[i][j] +=  
                a[i][k] * b[k][j];  
    }  
}
```

包含很多“位运算”

C语言：浮点数相乘



```
1 — clear;  
2 — clc;  
3 — a = randn (3,4);  
4 — b = ones (4,5);  
5 — c = a*b;
```

工作区

名称 ^	值
 a	3x4 double
 b	4x5 double
 c	3x5 double

包含很多“浮点数相乘”

MATLAB语言：矩阵相乘



为什么需要数据类型

数据类型：元素的集合和定义在此集合上的一组操作的总称

（如C语言中的浮点型、MATLAB中的矩阵）

- 一种封装好的数据结构
- 实现了信息的隐藏，把用户无须了解的细节封装在类型中
- 刻画了操作对象的外在特性
- 使用时，不必知道计算机内部的组织和表示的细节，也不必知道所规定的操作是如何实现的

面向对象的程序设计：重用性、灵活性、扩展性

C++中的类：封装数据和操作

为什么需要数据类型



How to Grow a Mind: Statistics, Structure, and Abstraction
Joshua B. Tenenbaum *et al.*
Science **331**, 1279 (2011);
DOI: 10.1126/science.1192788



... it can infer that a hierarchical organization for the novel objects better fits the similarities people see in these objects...



数据类型 (Data Type)

- C语言中的基本数据类型：字符型, 整型, 浮点型, 双精度浮点和无值类型
- 程序中任何常量，变量都必须先定义类型 (类声明→定义对象)
- 定义的背后是**对关键属性和操作的约定**：存储位数，取值范围，
+，-，*，/，**%**，++，--（不同类型不一样的运算）
- 高级语言中的数据类型分为原子类型和结构类型
 - 原子类型：语言本身已经定义好的
 - **结构类型：用户可以自行定义的 → 抽象数据类型（ADT）**



抽象数据类型 – 复数

ADT Complex {

数据对象: $D = \{e1, e2 \mid e1, e2 \in \mathbb{R}\}$

数据关系: $R = \{\langle e1, e2 \rangle \mid e1 = \text{Real}(D), e2 = \text{Imag}(D)\}$

基本操作:

InitComplex(&Z, v1, v2)

操作结果: 构造复数Z, 其实部和虚部分别被赋以参数v1和v2的值。

DestroyComplex(&Z)

初始条件: 复数Z存在;

操作结果: 复数Z被销毁。

Add(z1, z2, &sum)

初始条件: z1, z2是复数。

操作结果: 用sum返回两个复数z1, z2的和值。

Sub(z1, z2, &sub)

初始条件: z1, z2是复数。

操作结果: 用sub返回两个复数z1, z2的差值。

} ADT Complex



抽象数据类型 – 圆柱体

ADT CYLinder {

数据对象: $D = \{r, h \mid r, h \in \mathbb{R}\}$

数据关系: $R = \{ \langle r, h \rangle \mid r \text{ 为圆柱底面半径}, h \text{ 为圆柱高} \}$

基本操作

InitCylid(r, h)

操作结果: 构造圆柱体, 底面半径 r , 圆柱高 h 。

BaseArea(r, &bArea)

初始条件: 圆柱体存在。

操作结果: 计算圆柱体底面积, 用 $bArea$ 返回。

SideArea(r, h, &sArea)

初始条件: 圆柱体存在。

操作结果: 计算圆柱体侧面积, 用 $sArea$ 返回。

Volume(r, h, &vol)

初始条件: 圆柱体存在。

操作结果: 计算圆柱体体积, 用 vol 返回。

}ADT CYLinder



抽象数据类型的描述

➤ 描述方法：

ADT 抽象数据类型名 {

数据对象：〈数据对象的定义〉

数据关系：〈数据关系的定义〉

基本操作：〈基本操作的定义〉

基本操作名（参数表）

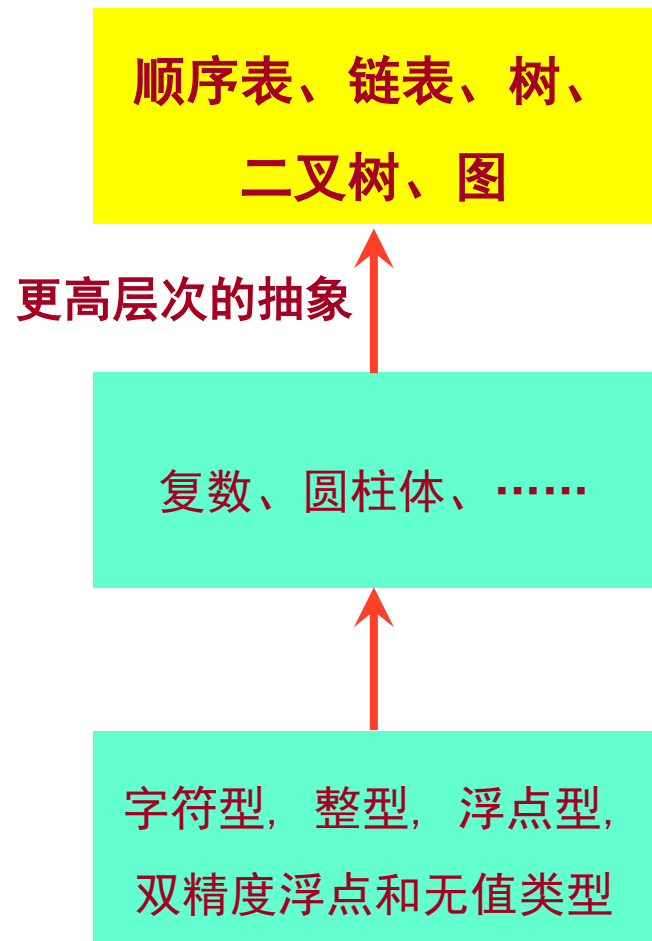
初始条件：〈初始条件描述〉

操作结果：〈操作结果描述〉

} ADT 抽象数据类型名

➤ 基本操作参数：

- 赋值参数提供输入值；
- 引用参数以&打头，用于返回操作结果





抽象数据类型 – 小结

抽象数据类型：一个数学模型（数据+关系）

以及定义在此数学模型上的一组操作

- 实现了数据抽象：描述实体的本质特征，功能及外部的用户接口
- 实现了数据封装：将实体的外部特性和内部实现细节分离，对外部用户隐藏内部实现细节，使得ADT的使用和实现分离
- 程序结构清晰，易于扩展，易于维护而不失其效率
- 提高了程序中数据安全性
- 大大增加了软件的复用程度



提纲

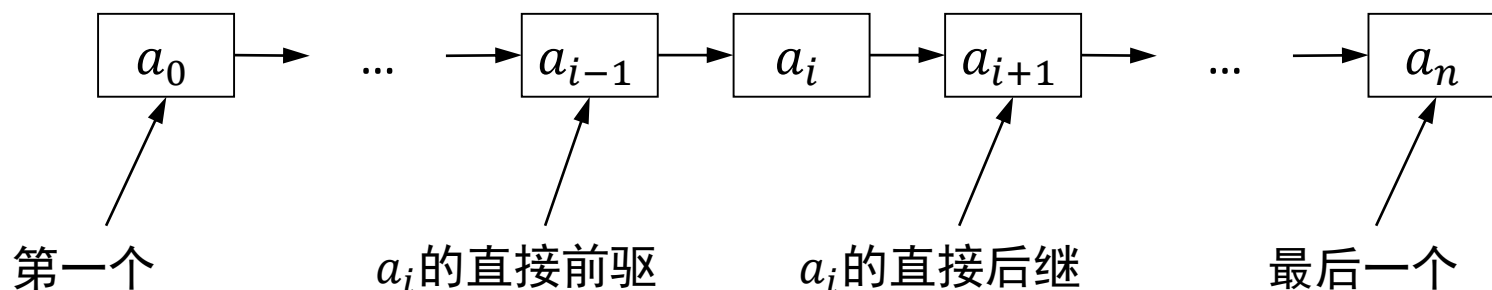
- 二元关系
- 抽象数据类型
- 线性表



线性表

线性表是一种“有序”结构，即在数据元素的非空有限集合中

- 存在唯一的一个被称为“第一个”的数据元素，无前驱；
- 存在唯一的一个被称为“最后一个”的数据元素，无后继；
- 除第一个之外，每个数据元素均只有一个直接前驱；
- 除最后一个之外，每个数据元素均只有一个直接后继；



“有序”是位序，数值上不一定有序



线性表

- 若将线性表记为 $(a_0, a_1, \dots, a_{i-1}, a_i, a_{i+1}, \dots, a_n)$
 - a_{i-1} 领先于 a_i , a_i 领先于 a_{i+1}
 - 称 a_{i-1} 是 a_i 的**直接前驱元素**; 称 a_{i+1} 是 a_i 的**直接后继元素**
 - 当 $i = 0, 1, 2 \dots n - 1$ 时, 有且仅有一个直接后继元素
 - 当 $i = 1, 2, 3 \dots n$ 时, 有且仅有一个直接前驱元素
- 线性表中元素的个数为线性表的**长度** (请多留意定量的参数)
 - 空表: 线性表为空, 长度为0
 - 非空表: 每个元素都有一个确定的位置, 称 i 为数据元素 a_i 在线性表中的**位序**: a_0 是第0个数据元素, a_{n-1} 是第 $n - 1$ 个数据元素, a_i 是第 i 个数据元素
- 同质: 线性表中元素可以是各种各样的, 如整型, 浮点型, 字符型, 也可以是用用户自己定义的数据类型, 但**同一线性表中的元素必定具有相同特性**



线性表的抽象数据类型 (ADT)

ADT List{

数据对象: $D = \{a_i | a_i \in ElemSet, i = 0, 1 \dots n, n \geq 0\}$

数据关系: $R = \{\langle a_{i-1}, a_i \rangle | a_i \in D, i = 1, 2 \dots n\}$

基本操作:

InitList(&L);

操作结果: 构造一个空的线性表L。

DestroyList(&L);

初始条件: 线性表已存在。

操作结果: 销毁线性表L。

IsEmpty(L);

初始条件: 线性表L已存在。

操作结果: 若L为空表, 则返回TRUE, 否则返回FALSE。



线性表的抽象数据类型 (ADT)

ListLength(L);

初始条件：线性表L已存在。

操作结果：返回L中数据元素个数。

GetElem(L, i, &e);

初始条件：线性表L已存在。

操作结果：用e返回L中第*i*个数据元素的值。

LocateElem(L, e, compare());

初始条件：线性表L已存在，compare()是数据元素判定函数。

操作结果：返回L中第1个与e满足关系compare()的数据元素的位置；若这样的数据元素不存在，则返回-1。



线性表的抽象数据类型 (ADT)

PriorElem(L,cur_e,&pre_e);

初始条件：线性表L已存在。

操作结果：若cur_e是L的数据元素，且不是第一个，则用pre_e返回它的前驱，否则操作失败，pre_e无定义。

NextElem(L,cur_e,&next_e);

初始条件：线性表L已存在。

操作结果：若cur_e是L的数据元素，且不是最后一个，则用next_e返回它的后继，否则操作失败，next_e无定义。

ClearList(&L);

初始条件：线性表L已存在。

操作结果：将L重置为空表。



线性表的抽象数据类型 (ADT)

ListInsert(&L,i,e)

初始条件：线性表L已存在， $0 \leq i \leq \text{ListLength}(L)$

操作结果：在L中第i个位置插入新的数据元素e，L的长度加1。

ListDelete(&L,i,&e)

初始条件：线性表L已存在， $0 \leq i \leq \text{ListLength}(L)-1$

操作结果：删除L的第i个数据元素，用e返回其值，L的长度减1。

ListTraverse(L,visit());

初始条件：线性表L已存在。

操作结果：依次对L的每个数据元素调用函数visit()。

} ADT List



线性表的抽象数据类型 (ADT)

➤ 基本操作的分类

- **构造型操作**: 构建一个线性表
- **销毁型操作**: 把已经存在的线性表销毁
- **引用型操作**: 得到线性表中的数据元素或线性表的某种特性
- **加工型操作**: 使线性表发生改变

- 以遍历为例, 可能是引用型, 也可能是加工性, 取决于操作visit()
- 对于其他的数据类型, 这些操作类似, 但实现方式会有很大差异; 我们关注这些操作在特定存储结构下的实现方式、复杂度分析等



线性表的操作举例：合并

假设有两个线性表La和Lb，要将两个线性表合并：将所有在线性表Lb中但不在La中数据元素插入到La中

```
void list_union(List &La, List Lb) {  
    int La_len = ListLength(La);           //求La长度  
    int Lb_len = ListLength(Lb);           //求Lb长度  
    for(i = 0; i < Lb_len; i++){  
        GetElem(Lb, i, &e);                //取Lb表中元素  
        if(LocateElem(La, e, equal) == -1){ //查访线性表La  
            ListInsert(&La, La_len, e);     //插入操作  
            La_len++;  
        }  
    }  
}
```



线性表的操作举例：保序合并

已知线性表La和Lb中的元素按值的递增顺序排列，要求将两个线性表归并成为一个新的有序线性表Lc

```
void list_merge(List La, List Lb, List &Lc) {  
    initList(&Lc);      i = 0; j = 0; k = 0;           //构造Lc  
    int La_len = ListLength(La); int Lb_len = ListLength(Lb); //求La和Lb长度  
    while(i < La_len && j < Lb_len){  
        GetElem(La,i,&ai);      GetElem(Lb,j,&bj);     //取元素  
        if(ai <= bj)      { ListInsert(&Lc,k,ai); k++;i++;} //ai入表  
        else { ListInsert(&Lc,k,bj); k++;j++;}           //bj入表  
    }  
    while(i < La_len) { GetElem(La, i, &ai); ListInsert(&Lc,k,ai); i++; k++; }  
    while(j < Lb_len) { GetElem(Lb, j, &bj); ListInsert(&Lc,k,bj); j++; k++; }  
}
```



线性表的顺序存储：顺序表

➤ 线性表的顺序存储：在存储器中用一组地址连续的存储单元存储线性表的数据元素，得到顺序表

➤ 顺序表中的元素存储关系

- 基地址：第一个元素 a_0 的存储位置
- 假设每个元素占用 l 个存储单元，则存储地址

$$LOC(a_{i+1}) = LOC(a_i) + l$$

$$LOC(a_i) = LOC(a_0) + i \times l$$

存储地址可根据位序确定

存储地址	内存状态	位序
b	a_0	0
$b + l$	a_1	1
	...	...
$b + i \times l$	a_i	i
$b + (i + 1) \times l$	a_{i+1}	$i + 1$
	...	...
$b + (n - 1) \times l$	a_{n-1}	$n - 1$



顺序表的抽象数据类型（C++实现）

```
class SqList{
private:
    int elem[LIST_MAX_SIZE];           //存储空间（静态分配）
    int length;                        //顺序表的长度
public:
    SqList() {length = 0;}             //空表表长设为0
    void clearSqList() {length = 0;}   //清空线性表L
    bool IsEmpty() {return length == 0;} //判断是否为空表
    int ListLength() {return length;}  //返回表长
    void GetElem(int l, int *e) {*e = elem[l];}
    int LocateElem(int e);
    void PriorElem(int cur_e, int *pre_e);
    void NextElem(int cur_e, int *next_e);
    void ListInsert(int i, int e);
    void ListDelete(int i, int *e);
    void ListTraverse(bool (*visit)(int e)); //具体实现参考附录
}
```



顺序表的主要操作 – 插入操作

- 插入操作：在顺序表的第 i 个位置插入一个新元素，使顺序表的长度从 n 增加到 $n + 1$
 - 插入前： $(a_0, a_1, \dots, a_{i-1}, a_i, a_{i+1}, \dots, a_{n-1})$
 - 插入后： $(a_0, a_1, \dots, a_{i-1}, \hat{a}, a_i, a_{i+1}, \dots, a_{n-1})$
- 复杂度分析
 - 基本操作：移动数据元素
 - 在顺序表的第 i 个位置插入一个新元素，需要移动 $n - i$ 个元素；
 - 假设从顺序表的第 i 个位置插入元素的先验概率为 p_i
 - 插入操作移动元素次数的期望为： $E_{insert} = \sum_{i=0}^n (n - i) \times p_i$



顺序表的主要操作 – 删除操作

- 删除操作：把顺序表的第 i 个位置的元素从表中删除，使长度为 n 的顺序表的长度变为 $n - 1$
 - 删除前： $(a_0, a_1, \dots, a_{i-1}, a_i, a_{i+1}, \dots, a_{n-1})$
 - 删除后： $(a_0, a_1, \dots, a_{i-1}, a_{i+1}, \dots, a_{n-1})$
- 复杂度分析
 - 基本操作：移动数据元素
 - 把顺序表的第 i 个位置上的元素删除，需要移动 $n - i - 1$ 个元素；
 - 假设从顺序表的第 i 个位置删除元素的先验概率为 q_i
 - 删除操作移动元素次数的期望为： $E_{delete} = \sum_{i=0}^{n-1} (n - i - 1) \times q_i$



顺序表主要操作的复杂度分析

- 不失一般性，我们假设在顺序表的任何位置插入或者删除元素的概率是相等的，因此有： $p_i = 1/(n+1)$, $q_i = 1/n$

$$\square E_{insert} = \frac{1}{n+1} \sum_{i=0}^n (n-i) = \frac{n}{2} \quad O(n)$$

$$\square E_{delete} = \frac{1}{n} \sum_{i=0}^{n-1} (n-i-1) = \frac{n-1}{2}$$

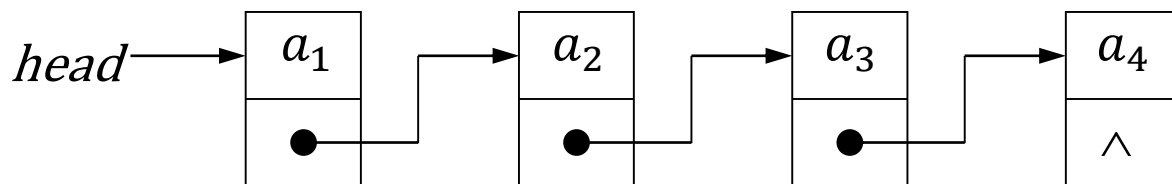
- 在顺序表中插入或者删除一个元素，平均移动表中大约一半元素

顺序表：引用型操作复杂度低、加工型操作复杂度高
“好找，但轻易别动”



线性表的链式存储：链表

- 逻辑上相邻的数据元素在存储地址上不需要相邻
- 依靠**附加指针**将逻辑上相邻的元素连接起来
- 每个数据元素包含**数据和指针**
- **单向链表**：链表结点 (Node) 由两个域组成
 - 数据域：存储数据元素
 - 指针域：指向直接后继结点



```
结构体定义链表结点  
typedef struct node {  
    int data;  
    struct node *next;  
}NODE;
```



单向链表的抽象数据类型（C++实现）

```
class LinkList{
private:
    NODE *head;                                     //单向链表的头指针
public:
    LinkList() {head = NULL;}                       //构造单向链表
    ~LinkList();                                    //销毁单向链表
    bool    clearSqList();                           //清空单向链表
    bool    IsEmpty() {return head == NULL;}         //判断单向链表表长是否为0
    int     Length();                                //求单向链表的表长
    bool    GetElem(int i, int *e);                  //取单向链表的元素
    int     LocateElem(int e);                        //定位链表中的元素位置
    bool    PriorElem(int cur_e, int *pre_e);        //取上一个元素
    bool    NextElem(int cur_e, int *next_e);        //取下一个元素
    bool    Insert(int i, int e);                     //向单向链表中插入元素
    bool    Delete(int i, int *e);                    //删除单向链表中的元素
    bool    Traverse(bool (*visit)(int e));          //遍历单向链表
};
```



单向链表的主要操作 – 定位元素

链表中不存在“位序”的概念，无法直接计算得到元素的存储位置，
需通过多次指针的定向来得到

```
bool LinkList::GetElem(int i, int *e){  
    NODE *p = head;  
    int j = 0;  
    while(p && j < i){  
        p = p->next;  
        j++;  
    }  
    if(p == NULL)    return FALSE;  
    *e = p->data;  
    return TRUE;  
}
```

//取链表中第i个位置的元素的值
//辅助指针用于标示当前查找位置
//辅助变量用于标示当前位序

//p指向直接后继

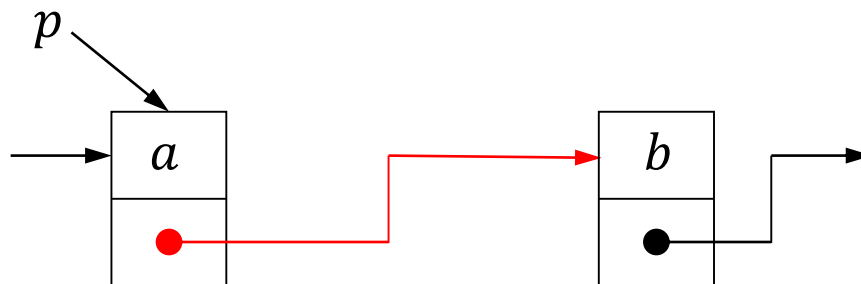
//未找到，返回错误
//返回要查找的元素



单向链表的主要操作 – 插入元素

在结点 a 之后插入元素 x

➤ 定位结点的位置

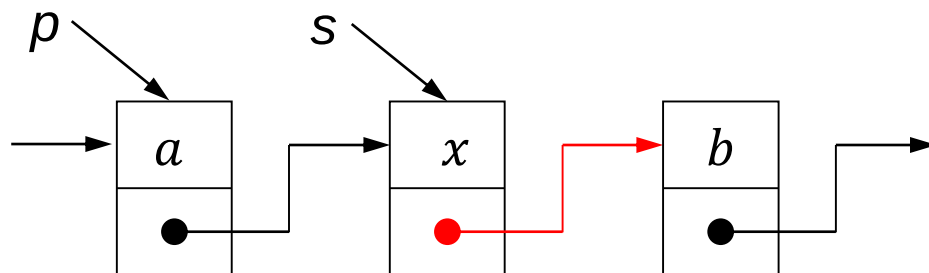


➤ 插入操作

❑ $s \rightarrow \text{next} = p \rightarrow \text{next};$

❑ $p \rightarrow \text{next} = s;$

指针操作原则：先继承、再求新





单向链表的主要操作 – 插入元素

```
bool LinkedList::Insert(int i, int e){  
    NODE *p = head, *s;  
    int j = 0;  
    while(j < i - 1){ p = p->next; j++;}  
    s = (NODE *)new NODE[1];  
    s->data = e;  
    s->next = p->next;  
    p->next = s;  
    return TRUE;  
}
```

//定位插入位置

//为新结点分配内存

//插入值赋给新元素

//插入结点

空链表、头结点插入、...





单向链表的主要操作 – 插入元素

```
bool LinkList::Insert(int i, int e){  
    NODE *p = head, *s;    int j = 0;  
    if(i == 0) {  
        s = (NODE *)new NODE[1];  
        s->data = e;  
        s->next = p;  
        head = s;    return TRUE;  
    }  
    while(p && j < i - 1){ p = p->next; j++;}  
    if(!p ) return FALSE;  
    s = (NODE *)new NODE[1];  
    s->data = e;  
    s->next = p->next; p->next = s;  
    return TRUE;  
}
```

//头结点插入

//为新结点分配内存

//给新结点赋值

//插入结点

//判断不是空表并定位插入位置

//找不到位置，返回错误

//为新结点分配内存

//插入值赋给新元素

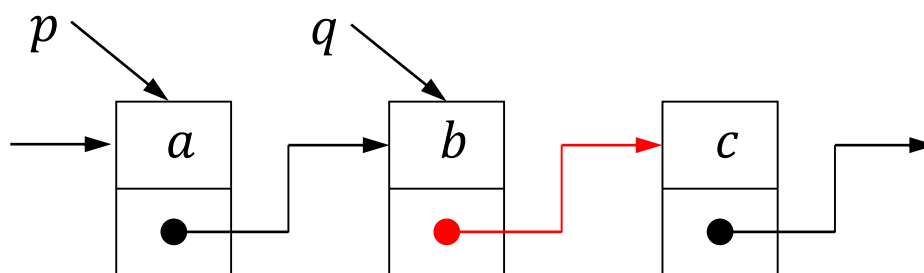
//插入结点



单向链表的主要操作 – 删除元素

删除结点 b

- 定位：令指针 p 指向 a 所在结点，指针 q 指向 b 所在结点



- 删除：
 $p \rightarrow \text{next} = q \rightarrow \text{next};$
delete $q;$

指针操作原则：先继承、再求新
注意：销毁 q ，防止内存泄漏





单向链表的主要操作 – 删除元素

```
bool LinkList::Delete(int i, int &e){  
    NODE *p = head, *q;      int j = 0;  
    if(!p)    return FALSE;  
    if(i == 0){  
        head = head->next;  
        e = p->data; delete p; p = NULL;  
        return TRUE;  
    }  
    while(p->next && j < i - 1){p = p->next; j++;}  
    if(!(p->next) || j > i-1) return FALSE;  
    q = p->next;  
    p->next = q->next;  
    e = q->data; delete q; q = NULL;  
    return TRUE;  
}
```

//空表不能删除

//删除头结点

//新的头结点

//取出元素，并删除结点

//定位删除位置

//未找到位置，返回错误

//取出带删除结点

//删除

//取出元素，并销毁结点



单向链表 vs. 顺序表

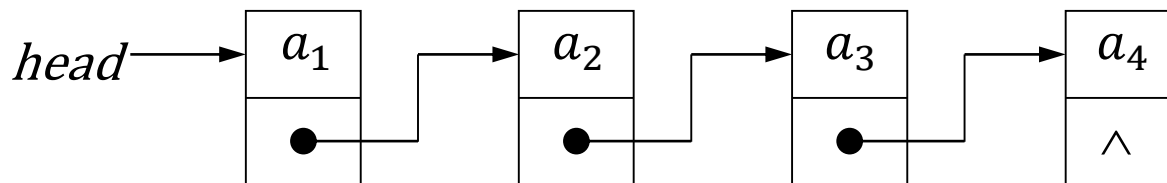
基本操作	功能	顺序表	单向链表
InitList(&L);	构造	$O(1)$	$O(1)$
DestroyList(&L);	销毁	$O(1)$	$O(n)$
IsEmpty(L);	判空	$O(1)$	$O(1)$
ListLength(L);	求长度	$O(1)$	$O(n)$
GetElem(L,i,&e);	按位序取元素		
LocateElem(L,e,compare());	定位元素	$O(n)$	$O(n)$
PriorElem(L,cur_e,&pre_e);	求前驱	$O(n)$	$O(n)$
NextElem(L,cur_e,&next_e);	求后继	$O(n)$	$O(n)$
ListInsert(&L,i,e)	按位序插入		
ListDelete(&L,i,&e)	按位序删除		
ClearList(&L);	清空	$O(1)$	$O(n)$
ListTraverse(L,visit());	遍历	$O(n)$	$O(n)$



单向链表的改进

➤ 单向链表是最简单的链表形式

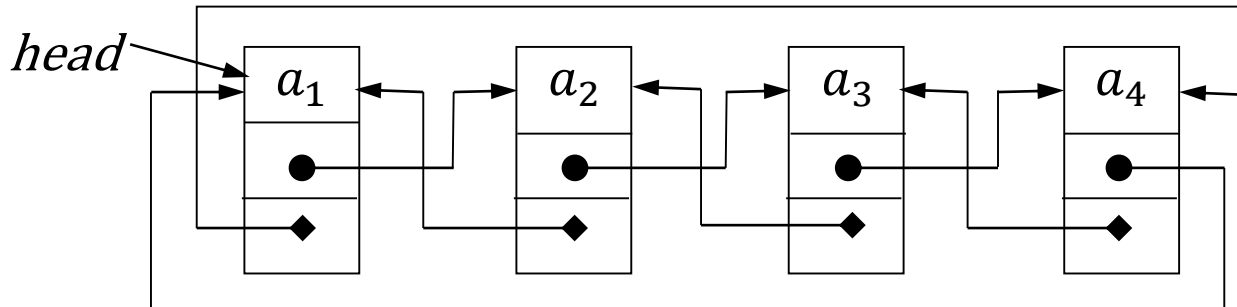
- 指针总是从前驱结点指向后继结点，访问后继结点容易，但是反过来访问前驱结点就要困难得多，需要从头开始依次访问各个结点，直到找到当前结点的前驱结点
- 获取表尾指针需要遍历整个链表，获得表长信息也是
- 插入或者删除元素时，需要在链表中依序寻找操作位置



➤ 改进：增加表长信息，增加尾指针，…，双向循环链表



双向循环链表



数据域:

data

指针域:

next

指针域:

prev

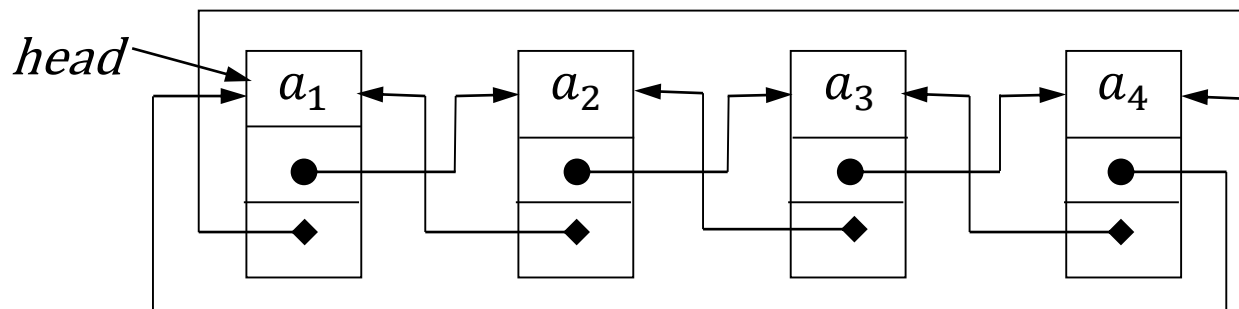
- 数据域：存储数据元素
- 指针域：
 - 前向指针：指向直接前驱结点
 - 后向指针：指向直接后继结点

结构体定义链表结点

```
typedef struct node    {  
    int data;  
    struct node *next;  
    struct node *prev;  
}NODE;
```



双向循环链表



数据域:

data

指针域:

next

指针域:

prev

➤ 优点

- ❑ 快速获得结点的直接前驱和直接后继
- ❑ 快速得到链表的尾结点

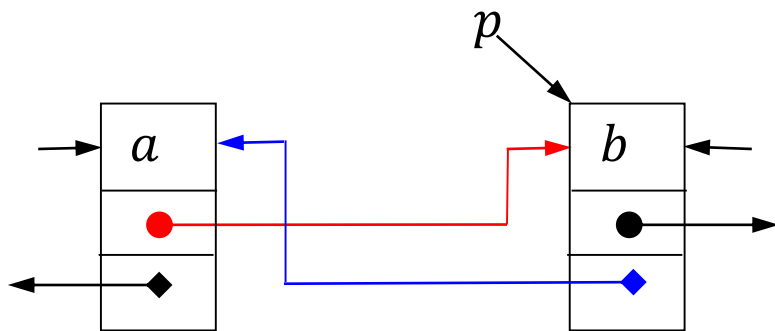
➤ 代价

- ❑ 额外空间
- ❑ 插入、删除操作需要同时维护两个方向的指针

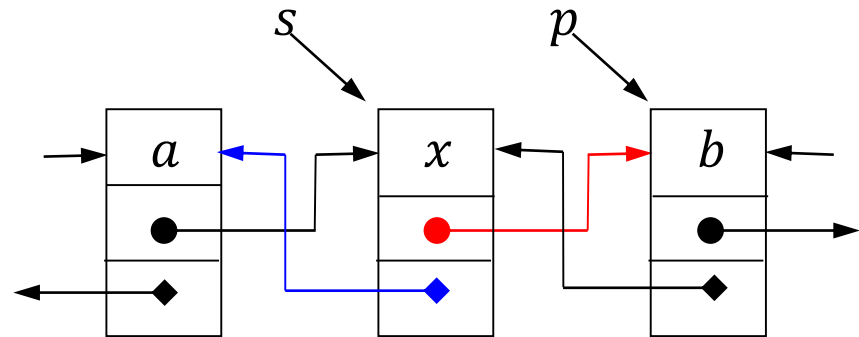


双向循环链表的主要操作 – 插入操作

在 p 指针所指结点前面插入元素 x



插入元素前



插入元素后

操作步骤:

```
s->prev = p->prev;  
p->prev->next = s;  
s->next = p;  
p->prev = s;
```

指针操作原则:
先继承、再求新



双向循环链表的主要操作 – 插入操作

```
bool C2LinkList::Insert(int i, int e){  
    NODE *p = head, *s; int j = 0;  
    if(!p) {  
        s = (NODE *)new NODE[1];  
        s->data = e; s->prev = s;  
        s->next = s; head = s;  
        return TRUE;  
    }  
    if(!(p = GetElemP(i))) return FALSE;  
    s = (NODE *)new NODE[1];  
    s->data = e;  
    s->prev = p->prev; p->prev->next = s;  
    s->next = p;      p->prev = s;  
    return TRUE;  
}
```

//空表

//为新结点分配内存

//结点赋值

//找不到插入位置，返回错误

//为新结点分配内存

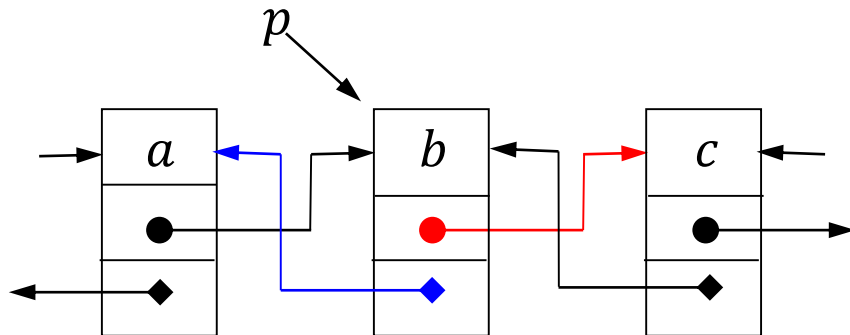
//插入元素赋值

//插入

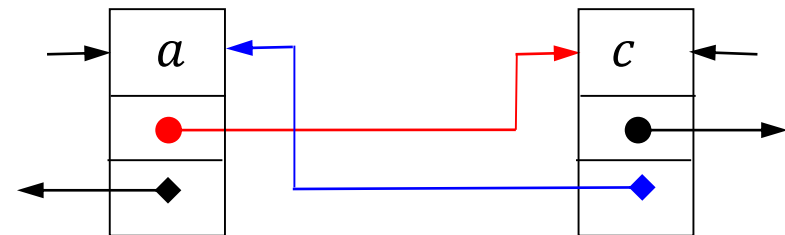


双向循环链表的主要操作 – 删除操作

删除结点 b



删除结点前



删除结点后

操作步骤:

- $p \rightarrow \text{prev} \rightarrow \text{next} = p \rightarrow \text{next};$
- $p \rightarrow \text{next} \rightarrow \text{prev} = p \rightarrow \text{prev};$
- `delete p`

指针操作原则：
先继承、再求新
销毁 q ，防止内存泄漏



双向循环链表的主要操作 – 删除操作

```
bool C2LinkList::Delete(int i, int *e){  
    NODE *p = head;  
    if(!p) return FALSE;  
    if(!(p = GetElemP(i))) return FALSE;  
    *e = p->data;  
    p->prev->next = p->next;  
    p->next->prev = p->prev;  
    if(p == head)    head = p->next;  
    delete p; p = NULL;  
    return TRUE;  
}
```

//空表不能删除

//找不到位置，返回错误

//取出删除元素的值

//删除操作

//删除头结点的情况



线性表小结：顺序表和链表

➤ 线性表的顺序存储：顺序表

- 用一组地址连续的存储单元依次存储线性表的数据元素
- 优点：可按“位序”快速访问
- 缺点：插入、删除操作需要移动表中的数据元素，事先确定表长（过小溢出、过大空间效率低）

➤ 线性表的链式存储：链表

- 用一组“任意”的存储单元(附加指针)存储表中的数据元素
- 优点：插入、删除操作无需移动表中的数据元素；空间利用率高
- 缺点：不能按“位序”快速访问



本讲小结

- 二元关系是描述数据之间关系的数学方法
- 抽象数据类型实现数据和操作的封装，支持程序设计的重用性、灵活性、扩展性
- 顺序表和链表
 - 抽象数据类型
 - 插入、删除方法、复杂度分析
 - 空表、头结点等特殊情况处理
 - 链表指针操作原则：先继承、再求新

加油同学们!





附件：顺序表的部分C语言实现

```
#define LIST_MAX_SIZE 100
```

```
typedef struct {
```

```
    int  elem[LIST_MAX_SIZE];
```

```
    int  length;
```

```
}SqList;
```

//线性表的分配存储空间

//存储空间基址（静态分配）

//顺序表的长度

```
void  InitList( SqList *L) {L->length = 0; }
```

//空表表长设为0

```
void  ClearList(SqList *L) {L->length = 0; }
```

//清空线性表L

```
bool  IsEmpty(SqList *L) {return L->length == 0;}
```

//判断表长是否为0

```
int    ListLength(SqList *L) {return L->length;}
```

//返回表长

```
void  GetElem(SqList *L, int i, int *e) {*e = L->elem[i];}
```



附件：顺序表的部分C语言实现

```
int LocateElem(SqList *L, int e) {  
    int i=0;  
    while(L->elem[i]!=e && i < L->length) i++;  
    return i;  
}
```

```
void PriorElem(SqList *L, int cur_e, int *pre_e) {  
    int index;  
    index = LocateElem(L, cur_e);  
    *pre_e = L->elem[index-1];  
}
```

//定位元素的位序

//没有前驱怎么办？

```
void NextElem(SqList *L, int cur_e, int *next_e) {  
    int index;  
    index = LocateElem(L, cur_e);  
    *next_e = L->elem[index+1];  
}
```

//定位元素的位序

//没有后继怎么办？



附件：顺序表的部分C语言实现

```
void ListTraverse(SqList *L, bool (*visit)(int e)) {           //遍历顺序表
    int i = 0;
    while ((i<L->length) && (visit(L->elem[i])!=TURE))
        i++;
}

bool visit(int e) {
    printf("%d ",e);                                           //打印访问到的元素
    return TURE;
}
```